

Source code:

```
#function for printing the solution
def print_solution(board):
    n= len(board)
    for row in range(n):
        for col in range(n):
            print("Q" if board[row]== col else ".", end= "")
            print("\n "+ "=" * (3*n) + "\n") #=====
#check if queen attack each other
def is_safe(board, row, col):
    #check for row, column and diagonal conflict
    for r in range(row):
        if board[r]== col or abs(board[r]-col)==abs(r-row):
            return False
    return True
#Place the queens backtracking
def solve_n_queen(board, row,n,solution):
    #place queen row by row backtrack if the position is not safe
    if row == n:
        solution.append(board[:]) #[:]= array slicing to save the
        solution
        return
    for col in range(n):
        if is_safe(board,row,col):
            board[row] = col
            solve_n_queen(board,row+1,n,solution)
            board[row]= -1 #barcktrack
```

```

#main function
def n_queens(n):
    board = [-1]*n #-1 represents there is no queen
    solution= []
    solve_n_queen(board,0,n,solution)
    return solution

if __name__ == "__main__":
    N = 4
    print(f"\n Solving {N} Queen problem \n")
    result = n_queens(N)
    print(f"Total soltuion found: {len(result)}\n")
    for i, sol in enumerate(result, 1):
        print(f"Solution{i}:")
        print_solution(sol)

```

Output:

```
Solving 4 Queen problem
```

```
Total soltuion found: 2
```

```
Solution1:
```

```
.Q..
```

```
=====
```

```
...Q
```

```
=====
```

```
Q...
```

```
=====
```

```
..Q.
```

```
=====
```

```
Solution2:
```

```
..Q.
```

```
=====
```

```
Q...
```

```
=====
```

```
Solution2:
```

```
..Q.
```

```
=====
```

```
Q...
```

```
=====
```

```
...Q
```

```
=====
```

```
.Q..
```

```
=====
```

```
C:\Users\Roshan\Desktop\4th-Sem GITHub>
```

Source code:

```
% -- Family facts --

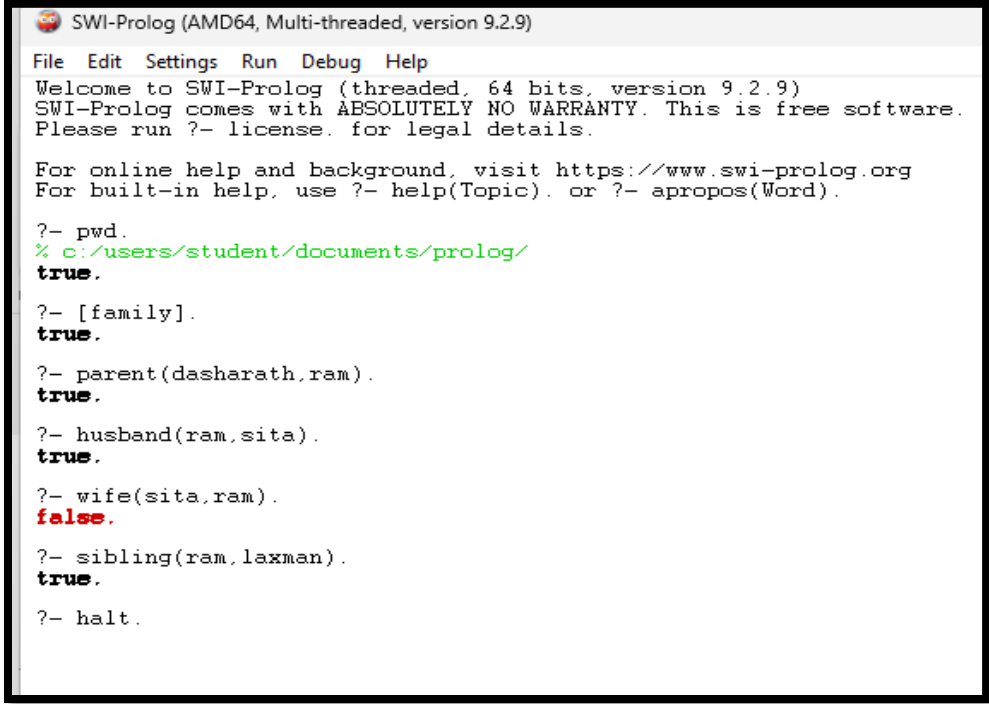
% Parent relationship
parent(dasharath,ram).
parent(dasharath,laxman).
parent(dasharath,bharat).
parent(dasharath,shatrughan).

% marriage
marriage(ram,sita).

% gender
male(ram).
male(dasharath).
male(laxman).
male(shatrughan).
female(sita).

% -- rules --
father(X,Y) :- parent(X,Y),male(X).
mother(X,Y) :- parent(X,Y),female(X).
sibling(X,Y) :- parent(P,X),parent(P,Y), X \= Y.
husband(X,Y) :- marriage(X,Y), male(X).
wife(X,Y) :- marriage(X,Y), female(X).
```

Output:



```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- pwd.
% c:/users/student/documents/prolog/
true.

?- [family].
true.

?- parent(dasharath,ram).
true.

?- husband(ram,sita).
true.

?- wife(sita,ram).
false.

?- sibling(ram,laxman).
true.

?- halt.
```

Source code:

```
:- use_module(library(clpfd)).  
solve_compact([S,E,N,D,M,O,R,Y]) :-  
    Vars = [S,E,N,D,M,O,R,Y],  
    Vars ins 0..9,  
    all_different(Vars),  
    S #\= 0,  
    M #\= 0,  
    1000*S + 100*E + 10*N + D  
+ 1000*M + 100*O + 10*R + E  
#= 10000*M + 1000*O + 100*N + 10*E + Y,  
    labeling([ff], Vars).
```

```
solve_carry([S,E,N,D,M,O,R,Y]) :-  
    Digits = [S,E,N,D,M,O,R,Y],  
    Carries = [C1,C2,C3,C4],  
    Digits ins 0..9,  
    Carries ins 0..1,  
    all_different(Digits),  
    S #\= 0,  
    M #\= 0,  
    D + E      #= Y + 10*C1,  
    N + R + C1 #= E + 10*C2,  
    E + O + C2 #= N + 10*C3,  
    S + M + C3 #= O + 10*C4,  
    C4 #= M,
```

```
labeling([ff], Digits).
```

```
print_solution([S,E,N,D,M,O,R,Y]) :-
```

```
SEND is 1000*S + 100*E + 10*N + D,
```

```
MORE is 1000*M + 100*O + 10*R + E,
```

```
MONEY is 10000*M + 1000*O + 100*N + 10*E + Y,
```

```
nl,
```

```
format("S = ~d, E = ~d, N = ~d, D = ~d~n", [S,E,N,D]),
```

```
format("M = ~d, O = ~d, R = ~d, Y = ~d~n", [M,O,R,Y]),
```

```
nl,
```

```
format("SEND = ~d~n", [SEND]),    #d= datatype
```

```
format("MORE = ~d~n", [MORE]),
```

```
format("MONEY = ~d~n", [MONEY]),
```

```
nl.
```

Output:

```
SWI-Prolog (AMD64, Multi-threaded, version 9.2.9)
File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- pwd.
% c:/users/student/documents/prolog/
true.

?- [crypto].
true.

?- solve_compact(Vars), print_solution(Vars).

S = 9, E = 5, N = 6, D = 7
M = 1, O = 0, R = 8, Y = 2

SEND = 9567
MORE = 1085
MONEY = 10652

Vars = [9, 5, 6, 7, 1, 0, 8, 2] ■
```